

CO1 AT1

ARTIFICIAL INTELLIGENCE ASSESSMENT

NAME:Anushka Sree Raj

REGNO:192571024

DATE:07-10-2026

ANSWERS:

(a) PEAS description of the warehouse delivery robot:

PEAS stands for Performance Measure, Environment, Actuators, and Sensors. It describes the task environment in which an intelligent agent operates.

P — Performance Measure

- Successfully deliver the package to cell (5,5).
- Minimize the number of moves.
- Avoid blocked cells and collisions.
- Reach the destination with minimum total cost.

E — Environment

- A 5×5 warehouse grid containing 25 cells.
- Initial position: (1,1).
- Goal position: (5,5).
- Blocked cells: (2,2), (2,3), (3,3), (4,2), and (4,4).
- The robot can move only horizontally or vertically.

A — Actuators

- Move Up.
- Move Down.
- Move Left.
- Move Right.
- Pick up or release the package, if delivery actions are included in the physical system.

S — Sensors

- Position sensors to identify the current grid cell.
- Obstacle sensors to detect shelving units.

- Encoders or motion sensors to track movement.
- A destination or task input identifying the delivery location.

The PEAS description specifies how the robot measures success, where it operates, how it moves, and what information it uses to navigate safely.

(b) Classification of the intelligent agent

The most suitable classification is a goal-based agent.

A goal-based agent selects actions by considering the desired goal state and determining which sequence of actions will reach it.

Justification:

- The robot has a clearly defined goal: reach cell (5,5) from cell (1,1).
- It must consider possible routes through the grid.
- It must avoid the five blocked cells while choosing its movements.
- It can use search algorithms such as BFS or DFS to find a route.
- Since every move costs 1, BFS can find a minimum-cost route to the destination.

other agent types:

- Simple reflex agent: Selects actions based on the current percept and condition-action rules. It does not necessarily plan a complete route.
- Model-based agent: Maintains an internal representation of the environment. This can help the robot remember obstacles, but a model alone does not specify the delivery goal.
- Utility-based agent: Chooses actions by comparing their utility, such as balancing travel time, battery consumption, and safety. This is useful when several competing objectives exist.
- Goal-based agent: Specifically plans actions to reach the required destination, making it the best fit for this problem.

A goal-based agent is suitable because the robot must plan a path that reaches a specified destination while avoiding obstacles.

(c) Formal search problem formulation :

A search problem consists of five components: Initial State, Goal State, Actions, Transition Model, and Path Cost.

Component	Formal definition
Initial State	$S_0 = (1,1)$

Goal State	$G = (5,5)$
Actions	Up, Down, Left, Right
Transition Model	An action moves the robot to an adjacent cell if that cell is inside the grid and is not blocked.
Path Cost	Each move costs 1; the total path cost is the number of moves.

State space: All unblocked cells in the 5×5 grid.

Blocked cells:

$(2,2), (2,3), (3,3), (4,2), (4,4), (2,2), (2,3), (3,3), (4,2), (4,4), (2,2), (2,3), (3,3), (4,2), (4,4).$

Transition function:

$$T((r,c),a)=(r',c') \quad T((r,c),a)=(r',c') \quad T((r,c),a)=(r',c')$$

where (r',c') is the adjacent cell resulting from action a , provided it is valid and unblocked. The robot cannot move outside the grid or enter a blocked cell.

Path cost function:

$$g(n) = \sum_{i=1}^n 1 = n \quad g(n) = \sum_{i=1}^n 1 = n \quad g(n) = \sum_{i=1}^n 1 = n$$

where n is the number of moves taken along the path.

(d) Breadth-First Search (BFS):

BFS explores the search tree level by level, visiting states with fewer moves before states requiring more moves. It uses a FIFO (First In, First Out) queue.

For this solution, we use the following conventions:

- Neighbor order: Up $\rightarrow$ Down $\rightarrow$ Left $\rightarrow$ Right.
- A cell is marked visited when it is added to the queue, preventing duplicate entries.
- The leftmost element of each queue is the next state to be expanded.
- Stop when the goal is removed from the queue.

The queue below is shown after each node is expanded, with the next node to process on the left.

Step	Expanded node	Newly added nodes	Queue after expansion (front $\rightarrow$ rear)
0	—	—	[(1,1)]
1	(1,1)	(2,1), (1,2)	[(2,1), (1,2)]
2	(2,1)	(3,1)	[(1,2), (3,1)]
3	(1,2)	(1,3)	[(3,1), (1,3)]
4	(3,1)	(4,1), (3,2)	[(1,3), (4,1), (3,2)]
5	(1,3)	(1,4)	[(4,1), (3,2), (1,4)]
6	(4,1)	(5,1)	[(3,2), (1,4), (5,1)]

7	(3,2)	None	[(1,4), (5,1)]
8	(1,4)	(2,4), (1,5)	[(2,4), (1,5), (5,1)]
9	(2,4)	(3,4), (2,5)	[(1,5), (5,1), (3,4), (2,5)]
10	(1,5)	None	[(5,1), (3,4), (2,5)]
11	(5,1)	(5,2)	[(3,4), (2,5), (5,2)]
12	(3,4)	(3,5)	[(2,5), (5,2), (3,5)]
13	(2,5)	None	[(5,2), (3,5)]
14	(5,2)	(5,3)	[(3,5), (5,3)]
15	(3,5)	(4,5)	[(5,3), (4,5)]
16	(5,3)	(5,4)	[(4,5), (5,4)]
17	(4,5)	(5,5)	[(5,4), (5,5)]
18	(5,4)	None	[(5,5)]
19	(5,5)	Goal reached	[]

Step 3: Final BFS path

Tracing the parent of each state from the goal back to the start gives:

SHORTEST PATH FOUND BY BFS:

$(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$

Total moves:8

Total path cost:8

BFS finds an optimal path with a total cost of 8 because every move costs 1 and no shorter path reaches the goal.

Important correction to the BFS trace above: With the stated Up $\rightarrow$ Down $\rightarrow$ Left $\rightarrow$ Right neighbor order, the queue from Step 8 onward should be as follows. This ordering also determines the exact shortest path returned by BFS.

(e) Depth-First Search (DFS) :

DFS explores one branch as deeply as possible before backtracking. It uses a LIFO (Last In, First Out) stack.

For consistency, we use the same neighbour order: Up $\rightarrow$ Down $\rightarrow$ Left $\rightarrow$ Right. To make DFS explore neighbours in that order, valid neighbours are pushed onto the stack in reverse order. The rightmost entry below is the top of the stack.

Step-by-step DFS expansion:

Step	Expanded node	Newly added nodes	Stack after expansion (bottom $\rightarrow$ top)
0	—	—	[(1,1)]
1	(1,1)	(1,2), (2,1)	[(1,2), (2,1)]
2	(2,1)	(3,1)	[(1,2), (3,1)]
3	(3,1)	(3,2), (4,1)	[(1,2), (3,2), (4,1)]
4	(4,1)	(5,1)	[(1,2), (3,2), (5,1)]

5	(5,1)	(5,2)	[(1,2), (3,2), (5,2)]
6	(5,2)	(5,3)	[(1,2), (3,2), (5,3)]
7	(5,3)	(4,3), (5,4)	[(1,2), (3,2), (5,4), (4,3)]
8	(4,3)	None	[(1,2), (3,2), (5,4)]
9	(5,4)	(5,5)	[(1,2), (3,2), (5,5)]
10	(5,5)	Goal reached	[(1,2), (3,2)]

Explanation:

1. DFS starts at (1,1) and explores downward through (2,1), (3,1), and (4,1).
2. It continues along the bottom row through (5,1), (5,2), and (5,3).
3. From (5,3), it explores (4,3), which is a dead end because its other adjacent cells are blocked or already visited.
4. DFS backtracks to (5,3), then explores (5,4) and reaches the goal (5,5).

Final DFS path:

$(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$

- Total moves: 8
- Total path cost: 8

DFS happens to find an optimal path in this particular traversal, but DFS does not guarantee an optimal solution in general.

(f) Comparison of BFS and DFS:

Criterion	BFS	DFS
Completeness	Complete for a finite graph when visited states are tracked.	Complete for a finite graph when visited-state tracking and backtracking are handled correctly.
Optimality	Optimal when every step has the same cost.	Not generally optimal; it may find a longer route first.
Time complexity	$O(V + E)$ for graph search using adjacency lists.	$O(V + E)$ for graph search using adjacency lists.
Space complexity	$O(V)$ worst case, because frontier and visited states are stored.	$O(V)$ worst case for graph search with visited-state storage; stack-only DFS can use less space.
Data structure	FIFO queue.	LIFO stack.
Behaviour in this scenario	Finds an optimal path with 8 moves.	Also finds an 8-move path under the specified neighbour order, after exploring a dead end.

Here, V is the number of states (vertices) and E is the number of valid transitions (edges).

Suitable algorithm: BFS is comparatively more suitable:

BFS is more suitable for this warehouse delivery problem because:

- Every movement has the same cost of 1.
- BFS explores paths in increasing order of the number of moves.
- It guarantees a shortest path in this unweighted grid.
- It systematically searches the finite state space.
- DFS also finds an 8-move path here, but a different neighbour order could cause it to find a longer path first.

Both BFS and DFS successfully reach (5,5) in this example. However, BFS is the better choice when the robot must minimize the number of moves and the total delivery cost.